

INTRODUCTION TO MACHINE LEARNING

Predictive models with scikit-learn

From data to predictions, and the API that connects them.

Guillaume · Probabl

PREDICTIVE MODELING

What is a predictive model for?

It learns from **past examples** a rule that predicts something we **don't know yet** from things we **can observe**.

CLASSIFICATION

Is this email spam?

Observe: words, sender, links
Predict: spam or not

REGRESSION

What will this flat sell for?

Observe: surface, rooms, location
Predict: a price

CLASSIFICATION

Will this customer leave?

Observe: usage, contract, tickets
Predict: churn or stay

VOCABULARY

Data as a table: samples and features

X · features			y · target
Surface m²	**Rooms**	**Year built**	**Price k€**
72	3	1998	410
45	2	1975	265
120	5	2010	690
58	3	1964	310
…	…	…	…

Illustrative values

Sample

One row: one observation, here one flat.

Feature

One column: a measured property we can observe.

Target

The value we want to predict.

VOCABULARY

Training and testing

All labeled samples, split once before learning



Training

The algorithm sees features **and** targets, and adjusts the model so that its predictions match the known answers.

Testing

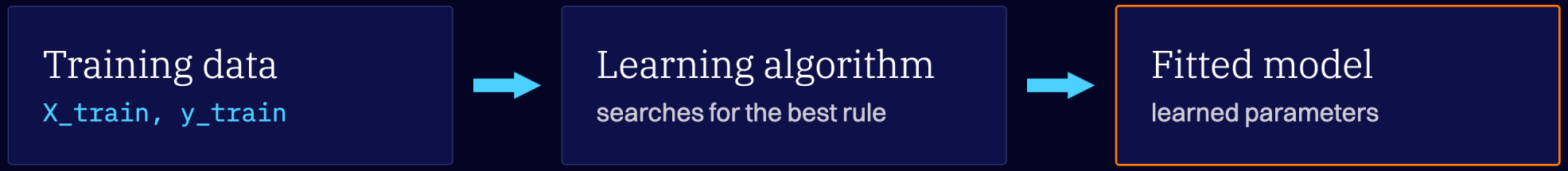
We predict on held-out samples the model never saw, and compare with their true targets.

Why hold data out? A model can memorize its training data. Only **unseen** data tells us how it will do on new cases.

THE FLOW

Build a model once, then use it

1 · CONSTRUCTION: TRAINING



2 · USAGE: PREDICTION



On the test set we also know y_test: comparing it with y_pred gives a **score**, our estimate of performance on new data.

SCIKIT-LEARN

Machine learning in Python

An open-source library, started in 2007, built on NumPy and SciPy, and maintained by a worldwide community.

Classification
predict a category

Regression
predict a number

Preprocessing
scale, encode, impute

Model selection
split, cross-validate, tune

```
pip install scikit-learn
```

Its key strength: **one consistent API** for every algorithm.

FROM FLOW TO API

Each step of the flow is one line of code

Choose a learning algorithm →

```
model = LogisticRegression()
```

Train on the training set →

```
model.fit(X_train, y_train)
```

Predict on new data →

```
y_pred = model.predict(X_new)
```

Evaluate on the test set →

```
model.score(X_test, y_test)
```

X: 2D array of shape (n_samples, n_features) · **y**: 1D array of shape (n_samples,) · the model is called an **estimator**

THE API IN PRACTICE

The whole flow in a few lines

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X, y = load_breast_cancer(return_X_y=True)  # samples × features, target
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=0)

model = LogisticRegression(max_iter=5000)  # choose
model.fit(X_train, y_train)  # train
y_pred = model.predict(X_test)  # predict
model.score(X_test, y_test)  # evaluate
```

ONE API, MANY MODELS

Switch the model, keep the code

```
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import HistGradientBoostingClassifier

models = [
    LogisticRegression(max_iter=5000),
    RandomForestClassifier(random_state=0),
    HistGradientBoostingClassifier(random_state=0),
]

for model in models:
    model.fit(X_train, y_train)
    print(model, model.score(X_test, y_test))
```

Same interface, different algorithm

Every estimator implements **fit**, **predict** and **score**. A linear model, a random forest or gradient boosting: the code around them does not change.

Scored on the **same test set**, the models can be compared fairly to pick the best one.



One flow. One API. Any model.

`.fit()` `.predict()` `.score()`

Docs and tutorials: scikit-learn.org